

Glimpse 初步产品设想

一、项目名称

Glimpse

含义：一瞥、快速看见。

Glimpse 希望表达的是：开发者不需要反复翻表结构、复制 DDL、回忆复杂 SQL 写法，就能快速看见数据查询路径和结果。

二、产品定位

Glimpse 是一个面向技术人员的 AI SQL 客户端。

它不是完整替代 DataGrip 的数据库 IDE，也不是主要面向业务人员的自然语言查数平台，而是一个小而精的开发者工具：

基于数据库上下文的 AI SQL 助手。

用户可以在 macOS 上安装 Glimpse，配置数据库连接信息，浏览数据库、选择表，然后通过自然语言描述查询需求，由 AI 基于表结构、字段注释、索引信息生成、修改、解释和修复 SQL。

三、目标用户

第一阶段主要面向：

- 后端开发
- 数据开发
- 测试开发
- 算法 / 策略研发
- 懂一些 SQL、但不想反复写复杂 SQL 的技术人员

暂时不主要面向纯业务人员。

原因是：技术人员通常知道自己要查什么，也知道大概涉及哪些表和字段，只是在复杂查询、SQL 方言、窗口函数、聚合、排序、分页、索引使用、报错修复等环节上会遇到效率问题。

四、核心问题

当前开发者在复杂查询场景中，经常会遇到以下问题：

1. 知道要查什么，但一时想不起 SQL 怎么写。
2. 需要反复查看表结构、字段类型、字段注释和索引。
3. 需要把 DDL、索引、字段含义复制给 AI，再让 AI 生成 SQL。
4. AI 生成 SQL 后，还需要复制回 DataGrip 执行。
5. 执行报错后，又要把错误信息贴回 AI 重新修复。

6. 多表查询、聚合统计、窗口函数、排序分页等场景容易写错。
7. 不确定生成的 SQL 是否能命中索引，是否存在性能问题。

Glimpse 要解决的不是“让完全不懂 SQL 的人查数”，而是：

帮助懂技术的人，在有数据库上下文的情况下，更快生成、修改、解释和修复 SQL。

五、产品边界

第一阶段不做大而全。

暂时不做

- 不做完整 DataGrip 替代品
- 不做复杂数据库 IDE
- 不做 ER 图建模
- 不做表结构设计和数据库迁移
- 不做数据编辑能力
- 不做 BI 报表系统
- 不做企业指标平台
- 不做复杂业务语义层
- 不做面向纯业务人员的 ChatBI
- 不一开始支持所有数据库
- 不一开始做自动选表和自动归因

第一阶段只聚焦

- 数据库连接配置
- 库表浏览
- 表结构读取
- 字段信息读取
- 索引信息读取
- 用户手动选择相关表
- 自然语言生成 SQL
- 基于已有 SQL 修改
- SQL 解释
- SQL 报错修复
- SQL 复制 / 后续可选执行

六、核心使用流程

第一版核心流程如下：

1. 用户安装 Glimpse macOS App。
2. 用户配置数据库连接信息。
3. Glimpse 读取数据库、表、字段、索引等元数据。
4. 用户选择一张或多张相关表。
5. 用户输入自然语言查询需求。
6. AI 基于当前表结构、字段注释和索引信息生成 SQL。
7. Glimpse 展示 SQL 和解释。
8. 用户复制 SQL 到其他工具执行，或在后续版本中直接在 Glimpse 内执行。

9. 如果 SQL 报错，用户贴入报错信息，Glimpse 自动修复 SQL。

七、典型使用场景

用户想查：

最近 1 小时每个 passage_id 的待领取、已领取、已提交数量，按总量倒序取前 100。

传统流程是：打开 DataGrip，找表结构，确认 status 枚举，确认 create_time 字段，确认索引，再手写 SQL。

Glimpse 的流程是：

1. 用户选择 `task_detail` 表。
2. Glimpse 自动读取字段、注释、索引。
3. 用户输入查询需求。
4. Glimpse 生成 SQL。
5. Glimpse 解释使用了哪些字段、过滤条件、聚合方式和排序逻辑。

八、核心能力

1. 表上下文自动收集

Glimpse 需要自动采集数据库上下文，包括：

- 数据库类型
- database / schema
- table
- column name
- column type
- nullable
- default value
- column comment
- primary key
- unique index
- normal index
- index column order
- create table DDL

这是 Glimpse 区别于普通 AI 聊天工具的核心。

普通 AI 需要用户手动复制表结构，而 Glimpse 应该自动拿到这些上下文。

2. 自然语言生成 SQL

用户用自然语言描述查询目标，Glimpse 根据当前选中的表和上下文生成 SQL。

第一版优先支持：

- 条件查询
- 排序

- limit
- group by
- count / sum / avg
- case when
- 多字段聚合
- 简单 join
- 时间范围过滤
- top N 查询

3. 基于已有 SQL 修改

真实写 SQL 通常不是一次完成，而是不断修改。

用户可以继续输入：

- 改成按天统计
- 加一个 business_case 条件
- 只看 status = 2
- 按总量倒序
- 加 limit 100
- 把 create_time 换成 last_update_time
- 增加一个删除率字段

Glimpse 应该基于当前 SQL 继续修改，而不是每次重新生成。

4. SQL 解释

Glimpse 需要解释生成的 SQL：

- 查询了哪张表
- 使用了哪些字段
- 过滤条件是什么
- 聚合逻辑是什么
- 排序逻辑是什么
- 返回多少条数据
- 是否使用了索引相关字段

SQL 解释的目标不是教学，而是让开发者快速确认 SQL 是否符合预期。

5. 报错修复

用户执行 SQL 后，如果出现报错，可以把错误贴回 Glimpse。

Glimpse 根据：

- 原始 SQL
- 报错信息
- 表结构
- 字段信息
- 数据库方言

自动修复 SQL。

常见报错包括：

- Unknown column
- Syntax error
- Function not found
- GROUP BY 错误
- 字段类型不匹配
- SQL 方言不兼容

6. 后续可扩展：SQL 执行

第一版可以先不做内置执行，只支持生成和复制 SQL。

后续版本再增加：

- 执行 SELECT
- 查询结果展示
- 查询超时
- 取消查询
- 最大返回行数限制
- 错误捕获
- 结果复制 / 导出

九、产品形态

Glimpse 计划做成一个 macOS App，可以安装在本地使用。

整体形态类似轻量数据库客户端：

左侧

数据库连接和对象树：

- 数据源
- database / schema
- table
- columns
- indexes

中间

核心工作区：

- 自然语言输入框
- SQL 编辑器
- SQL 生成结果
- SQL 修改对话

右侧

上下文面板：

- 当前选中表结构
- 字段说明
- 索引信息
- AI 对 SQL 的解释

底部

后续可扩展：

- 查询结果
- 执行日志
- 报错信息
- EXPLAIN 结果

十、技术选型

桌面框架

推荐使用：

Tauri 2

原因：

- 比 Electron 更轻量
- 更符合小而精工具定位
- 支持 macOS App 打包安装
- 后端使用 Rust，适合处理本地能力
- 后续可以扩展 Windows / Linux
- 比 SwiftUI 更适合快速构建复杂工具型 UI

前端框架

推荐使用：

React + TypeScript

原因：

- 适合复杂交互型工具
- 生态成熟
- 适合构建数据库树、SQL 编辑器、聊天面板、结果表格等界面
- 后续组件和状态管理选择丰富

UI 组件

推荐：

shadcn/ui + Tailwind CSS

目标风格：

- 干净
- 克制
- 高级
- 高信息密度
- 暗色模式友好
- 类似 Linear / Raycast / Cursor 的气质

SQL 编辑器

推荐：

CodeMirror 6

用于支持：

- SQL 高亮
- 快捷键
- 编辑器体验
- 后续自动补全
- SQL 格式化
- 错误标记

本地后端

推荐：

Rust

原因：

- Tauri 原生后端就是 Rust
- 适合处理数据库连接
- 适合处理本地文件、密钥、SQLite、查询任务
- 适合做查询超时、取消查询、结果流式返回等能力

本地存储

普通配置使用：

SQLite

保存：

- 连接名称
- host
- port
- database

- username
- 最近打开的表
- 查询历史
- SQL 草稿
- 用户偏好
- AI 对话历史

敏感信息使用：

macOS Keychain

保存：

- 数据库密码
- API Key
- SSH 密钥

原则：

SQLite 只保存非敏感配置，Keychain 保存敏感信息。

AI 接入

建议设计成 OpenAI-compatible API。

配置项包括：

- base_url
- api_key
- model
- temperature
- max_tokens

这样后续可以兼容：

- OpenAI
- DeepSeek
- Qwen
- 公司内部大模型
- Ollama
- LM Studio
- 其他兼容 OpenAI API 的模型服务

SQL 校验

第一版可以先做基础规则：

- 只允许 SELECT / WITH / EXPLAIN
- 禁止 INSERT / UPDATE / DELETE / DROP / ALTER / TRUNCATE / CREATE
- 默认 LIMIT
- 设置超时时间
- 设置最大返回行数

后续可以接入：

- sqlglot
- Rust SQL parser
- EXPLAIN 分析

十一、首批数据库支持

第一阶段建议不要支持太多数据库。

优先顺序：

1. MySQL / TiDB
2. StarRocks
3. PostgreSQL
4. ClickHouse

第一版建议先只支持 MySQL / TiDB。

原因：

- 使用场景明确
- 元数据读取方式成熟
- 和实际工作场景贴近
- 可以更快验证产品价值

十二、推荐版本路线

V0.1：最小可用版本

目标：跑通最小闭环。

功能：

- Tauri + React macOS App
- 配置 MySQL / TiDB 连接
- 保存连接配置
- 读取 database / table / column / index
- 用户选择表
- 输入自然语言查询需求
- 生成 SQL
- 展示 SQL
- 展示 SQL 解释
- 支持复制 SQL

V0.2：SQL 迭代能力

目标：让它真正成为 SQL 助手。

功能：

- 基于已有 SQL 修改
- SQL 解释增强
- SQL 报错修复
- 查询历史
- SQL 草稿
- 常用 Prompt 模板

V0.3：内置执行能力

目标：从“生成 SQL”变成“查询工具”。

功能：

- 执行 SELECT
- 展示查询结果
- 查询超时
- 取消查询
- 最大行数限制
- 错误捕获
- 结果复制
- CSV 导出

V0.4：查询优化能力

目标：提升专业性。

功能：

- EXPLAIN 分析
- 索引命中判断
- 慢查询提示
- SQL 格式化
- SQL 安全校验增强
- 索引优化建议

V1.0：开发者日常可用版本

目标：成为开发者日常 SQL 查询辅助工具。

功能：

- 多数据源管理
- StarRocks 支持
- PostgreSQL 支持
- 历史 SQL 召回
- 常用查询模板
- 团队共享配置
- 团队共享查询模板
- 可选业务术语库

十三、产品原则

原则一：程序员控制，AI 辅助

Glimpse 不应该让 AI 完全黑盒自动驾驶。

应该是：

AI 生成，程序员确认，程序员执行，AI 修复。

这样更符合开发者工具的信任模型。

原则二：上下文优先

Glimpse 的价值不在于“调用 AI”，而在于它掌握数据库上下文。

上下文包括：

- 数据库类型
- 表结构
- 字段类型
- 字段注释
- 索引
- 主键
- 唯一约束
- 当前 SQL
- 用户修改意图
- 执行报错

上下文越完整，SQL 生成质量越高。

原则三：小而精

第一版不追求大而全。

Glimpse 只需要先把一个小闭环做到好用：

选择表 → 读取上下文 → 描述查询需求 → 生成 SQL → 解释 SQL → 修改 / 修复 SQL。

原则四：安全默认

默认只处理只读查询。

- 不默认执行写操作
- 不默认执行危险 SQL
- 不直接存储明文密码
- 不把敏感信息暴露给模型
- 执行前让用户可见 SQL
- 用户始终保留最终控制权

十四、一句话总结

Glimpse 是一个面向技术人员的 macOS AI SQL 客户端。

它不试图替代 DataGrip，也不一开始做复杂 ChatBI，而是聚焦一个小而高频的开发者痛点：

当程序员知道自己想查什么，但不想反复翻表结构、复制 DDL、回忆复杂 SQL、处理报错和分析索引时，Glimpse 可以基于数据库上下文快速生成、修改、解释和修复 SQL。